

EXPERIMENT-08(HDOC)

C Programming

Question 1: Swap Two Numbers (With and Without a Third Variable)

Step 1: Understand the Problem Statement

- Input: Two numbers (a, b), and a menu choice selecting the swapping method.
- Output: The values of a and b before swapping, and after swapping, for the chosen method.
- This is a menu-driven program: Option 1 swaps using a third (temporary) variable; Option 2 swaps without using a third variable, using arithmetic operations; Option 3 exits the program.

Formula(s) used: Method 1: $\text{temp} = a; a = b; b = \text{temp};$ Method 2: $a = a+b; b = a-b; a = a-b;$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Opt 1: a=5, b=10	Before: a=5.00,b=10.00 After: a=10.00,b=5.00	Before: a=5.00,b=10.00 After: a=10.00,b=5.00	Pass
2	Opt 2: a=7, b=2	Before: a=7.00,b=2.00 After: a=2.00,b=7.00	Before: a=7.00,b=2.00 After: a=2.00,b=7.00	Pass
3	Opt 1: a=4, b=4	Before: a=4.00,b=4.00 After: a=4.00,b=4.00	Before: a=4.00,b=4.00 After: a=4.00,b=4.00	Pass

Step 3: Algorithm

1. Start the program.
2. Repeat steps 3 to 8 until the user chooses to exit.
3. Display the menu with options: (1) Swap using third variable, (2) Swap without third variable, (3) Exit.
4. Read the user's choice.
5. If choice = 1: read a and b, display values before swapping, swap using a temporary variable ($\text{temp} = a; a = b; b = \text{temp};$), display values after swapping.
6. If choice = 2: read a and b, display values before swapping, swap without a temporary variable ($a = a+b; b = a-b; a = a-b;$), display values after swapping.
7. If choice = 3: exit the loop and stop the program.
8. If choice is invalid, display an error message and show the menu again.
9. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for all 3 test cases produces results identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

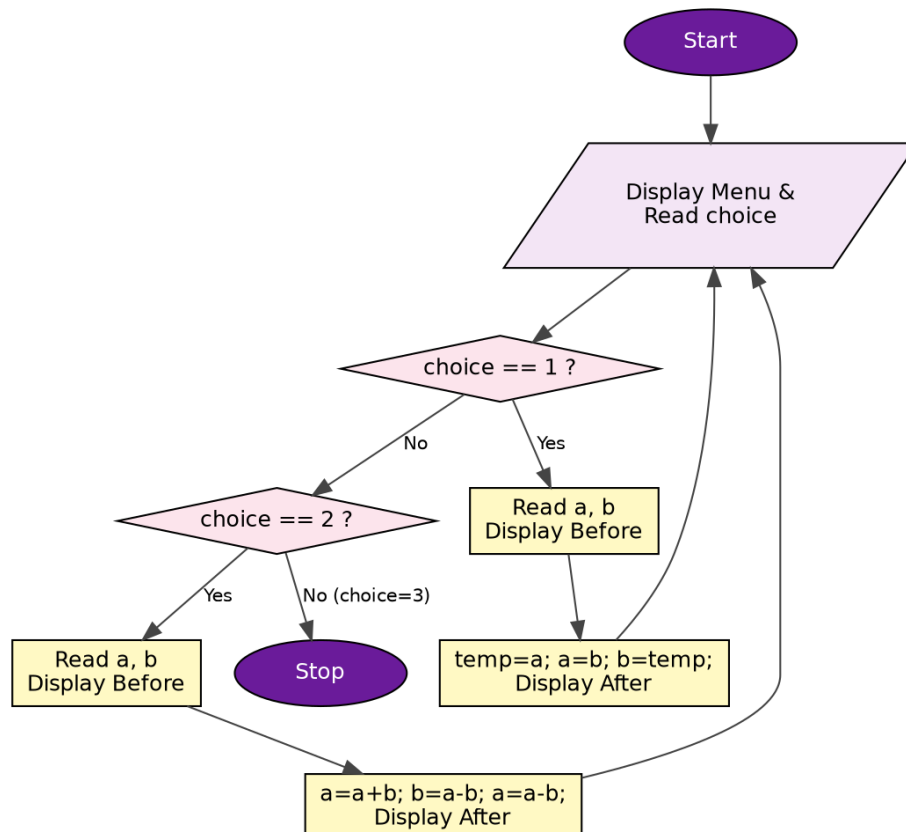
Step 4: Pseudocode

```

BEGIN
  REPEAT
    DISPLAY menu (1: third variable, 2: no third variable, 3: exit)
    READ choice
    IF choice = 1 THEN
      READ a, b
      PRINT "Before:", a, b
      temp <- a ; a <- b ; b <- temp
      PRINT "After:", a, b
    ELSE IF choice = 2 THEN
      READ a, b
      PRINT "Before:", a, b
      a <- a + b ; b <- a - b ; a <- a - b
      PRINT "After:", a, b
    ELSE IF choice = 3 THEN
      PRINT "Exiting"
    ELSE
      PRINT "Invalid choice"
    END IF
  UNTIL choice = 3
END

```

Step 5: Flowchart



Evaluation against Step 2 test cases: tracing each test case's menu choice and inputs through the decision boxes above yields the same results as the Expected Output(s) column, confirming the flowchart correctly represents the menu-driven logic.

Step 6: C Program

```
/* Q1: Menu-driven program to swap two numbers
Option 1: Swap using a third (temporary) variable
Option 2: Swap without using a third variable (arithmetic method)
Displays the values before and after swapping in both cases.
*/
#include <stdio.h>

int main() {
    int choice;
    float a, b, temp;

    while (1) {
        printf("\n--- SWAP MENU ---\n");
        printf("1. Swap using third variable\n");
        printf("2. Swap without using third variable\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice == 3) {
            printf("Exiting program...\n");
            break;
        }

        if (choice == 1 || choice == 2) {
            printf("Enter value for a: ");
            scanf("%f", &a);
            printf("Enter value for b: ");
            scanf("%f", &b);

            printf("\nBefore: a=%.2f,b=%.2f\n", a, b);

            if (choice == 1) {
                temp = a;
                a = b;
                b = temp;
            } else {
                a = a + b;
                b = a - b;
                a = a - b;
            }

            printf("After: a=%.2f,b=%.2f\n", a, b);
        } else {
            printf("Invalid choice! Please try again.\n");
        }
    }

    return 0;
}
```

}

Step 7: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc9.c          (compile)
./a.out             (execute)
```

All 3 test cases prepared in Step 2 were executed against the compiled program by selecting the corresponding menu option. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 8: Compilation and Execution Screenshot

```
Session  Actions  Edit  View  Help
(kali㉿kali)-[~]
$ nano hdoc9.c
(kali㉿kali)-[~]
$ cc hdoc9.c
(kali㉿kali)-[~]
$ ./a.out

— SWAP MENU —
1. Swap using third variable
2. Swap without using third variable
3. Exit
Enter your choice: 2
Enter value for a: 7
Enter value for b: 2

Before: a=7.00,b=2.00
After:  a=2.00,b=7.00

— SWAP MENU —
1. Swap using third variable
2. Swap without using third variable
3. Exit
Enter your choice: 1
Enter value for a: 5
Enter value for b: 10

Before: a=5.00,b=10.00
After:  a=10.00,b=5.00

— SWAP MENU —
1. Swap using third variable
2. Swap without using third variable
3. Exit
Enter your choice: 1
Enter value for a: 4
Enter value for b: 4

Before: a=4.00,b=4.00
After:  a=4.00,b=4.00
```

Question 2: Sum of First n Natural Numbers and Sum of Squares

Step 1: Understand the Problem Statement

- Input: A positive integer n, and a menu choice selecting which sum to compute.
- Output: Either the sum of the first n natural numbers, or the sum of squares of the first n natural numbers.
- This is a menu-driven program: Option 1 computes the sum of the first n natural numbers; Option 2 computes the sum of squares of the first n natural numbers; Option 3 exits the program.

Formula(s) used: $\text{Sum} = n(n+1)/2$ $\text{Sum of Squares} = n(n+1)(2n+1)/6$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Opt 1: n=10	Sum = 55	Sum = 55	Pass
2	Opt 2: n=5	Sum of squares = 55	Sum of squares = 55	Pass
3	Opt 1: n=1	Sum = 1	Sum = 1	Pass

Step 3: Algorithm

1. Start the program.
2. Repeat steps 3 to 6 until the user chooses to exit.
3. Display the menu with options: (1) Sum of first n natural numbers, (2) Sum of squares of first n natural numbers, (3) Exit.
4. Read the user's choice.
5. If choice = 1: read n, compute $\text{sum} = n(n+1)/2$, display sum.
6. If choice = 2: read n, compute $\text{sumSquares} = n(n+1)(2n+1)/6$, display sumSquares.
7. If choice = 3: exit the loop and stop the program.
8. If choice is invalid, display an error message and show the menu again.
9. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for all 3 test cases produces results identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: Pseudocode

```

BEGIN
  REPEAT
    DISPLAY menu (1: sum of n, 2: sum of squares, 3: exit)
    READ choice
    IF choice = 1 THEN
      READ n
      sum <- n * (n + 1) / 2
      PRINT sum
    ELSE IF choice = 2 THEN

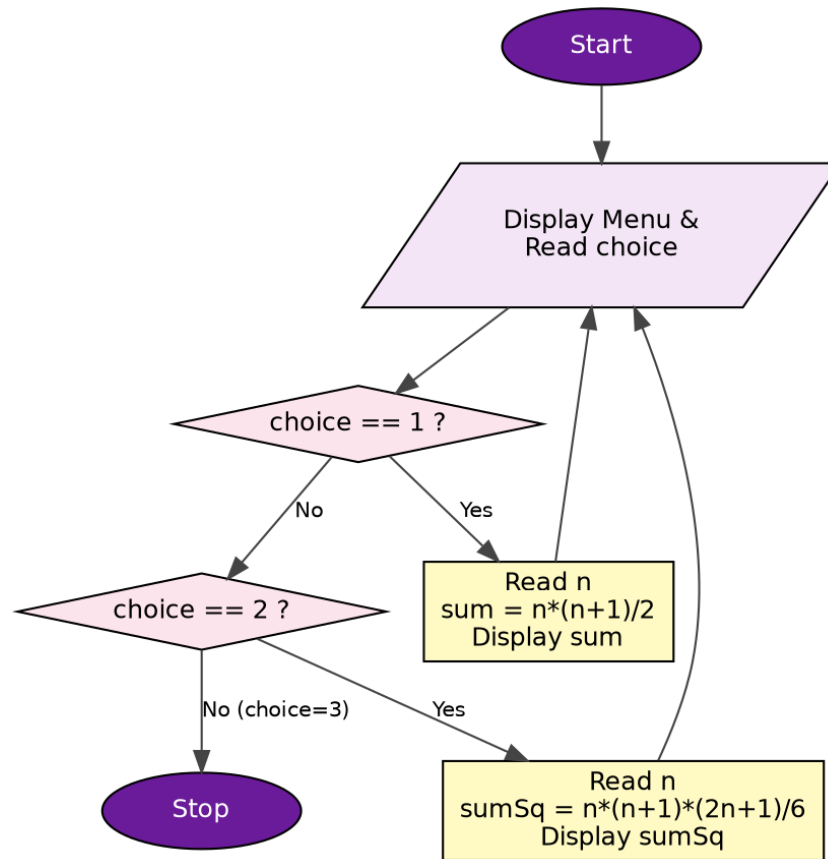
```

```

READ n
sumSquares <- n * (n + 1) * (2*n + 1) / 6
PRINT sumSquares
ELSE IF choice = 3 THEN
  PRINT "Exiting"
ELSE
  PRINT "Invalid choice"
END IF
UNTIL choice = 3
END

```

Step 5: Flowchart



Evaluation against Step 2 test cases: tracing each test case's menu choice and inputs through the decision boxes above yields the same results as the Expected Output(s) column, confirming the flowchart correctly represents the menu-driven logic.

Step 6: C Program

```

/* Q2: Menu-driven program
Option 1: Sum of first n natural numbers -> Sum = n*(n+1)/2
Option 2: Sum of squares of first n natural numbers -> Sum = n*(n+1)*(2n+1)/6
*/
#include <stdio.h>

int main() {
    int choice, n;

```

```

while (1) {
    printf("\n--- NATURAL NUMBERS MENU ---\n");
    printf("1. Sum of first n natural numbers\n");
    printf("2. Sum of squares of first n natural numbers\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    if (choice == 3) {
        printf("Exiting...\n");
        break;
    }

    if (choice == 1 || choice == 2) {
        printf("Enter n: ");
        scanf("%d", &n);

        if (choice == 1) {
            int sum = n * (n + 1) / 2;
            printf("Sum = %d\n", sum);
        } else {
            int sumSquares = n * (n + 1) * (2 * n + 1) / 6;
            printf("Sum of squares = %d\n", sumSquares);
        }
    } else {
        printf("Invalid option. Enter a valid choice.\n");
    }
}

return 0;
}

```

Step 7: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```

cc hdoc9_2.c      (compile)
./a.out           (execute)

```

All 3 test cases prepared in Step 2 were executed against the compiled program by selecting the corresponding menu option. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 8: Compilation and Execution Screenshot

Session Actions Edit View Help

```
(kali㉿kali)-[~]
$ nano hdoc9_2.c
```

```
(kali㉿kali)-[~]
$ cc hdoc9_2.c
```

```
(kali㉿kali)-[~]
$ ./a.out
```

— NATURAL NUMBERS MENU —

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
3. Exit

Enter your choice: 1

Enter n: 10 for a: 7

Sum = 55 for b: 2

— NATURAL NUMBERS MENU —

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
3. Exit

Enter your choice: 2

Enter n: 5 without using third variable

Sum of squares = 55

Enter your choice: 1

— NATURAL NUMBERS MENU —

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
3. Exit

Enter your choice: 1

Enter n: 1

Sum = 1

— NATURAL NUMBERS MENU —

1. Swap using third variable

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
3. Exit

Enter your choice: 3

Exiting...

Question 3: Simple Interest, Compound Amount, Compound Interest and Total Payable Amount

Step 1: Understand the Problem Statement

- Input: Principal (P), Rate of interest (R), Time (T), and a menu choice selecting which quantity to compute.
- Output: Simple Interest, Compound Amount, Compound Interest, or Total Payable Amount, depending on the option chosen.
- This is a menu-driven program with four calculation options plus an exit option: (1) Simple Interest, (2) Compound Amount, (3) Compound Interest, (4) Total Payable Amount (Principal + Simple Interest), (5) Exit.

Formula(s) used: $SI = (P \times R \times T) / 100$; $CA = P \times (1 + R/100)^T$; $CI = CA - P$; $Total Payable = P + SI$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Opt 1: P=1000,R=5,T=2	SI = 100.00	SI = 100.00	Pass
2	Opt 3: P=1000,R=5,T=2	CI = 102.50	CI = 102.50	Pass
3	Opt 4: P=2000,R=10,T=3	Total = 2600.00	Total = 2600.00	Pass

Step 3: Algorithm

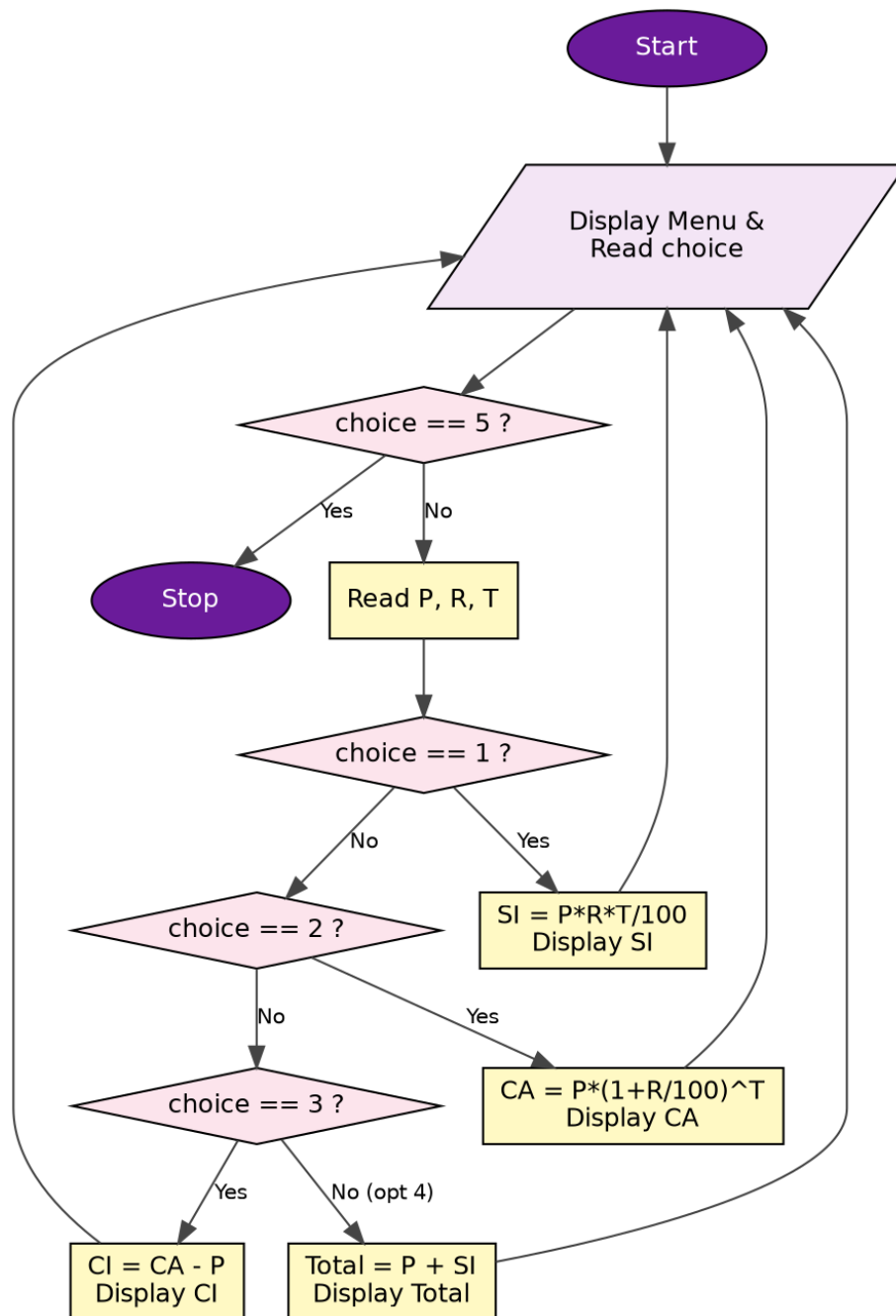
1. Start the program.
2. Repeat steps 3 to 9 until the user chooses to exit.
3. Display the menu with options: (1) Simple Interest, (2) Compound Amount, (3) Compound Interest, (4) Total Payable Amount, (5) Exit.
4. Read the user's choice.
5. If choice is between 1 and 4, read Principal (P), Rate (R) and Time (T).
6. If choice = 1: compute $SI = (P \times R \times T) / 100$ and display SI.
7. If choice = 2: compute $CA = P \times (1 + R/100)^T$ and display CA.
8. If choice = 3: compute CA, then $CI = CA - P$, and display CI.
9. If choice = 4: compute SI, then $Total = P + SI$, and display Total.
10. If choice = 5: exit the loop and stop the program.
11. If choice is invalid, display an error message and show the menu again.
12. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for all 3 test cases produces results identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: Pseudocode

```
BEGIN
  REPEAT
    DISPLAY menu (1:SI 2:CA 3:CI 4:Total 5:Exit)
    READ choice
    IF choice >= 1 AND choice <= 4 THEN
      READ P, R, T
    END IF
    IF choice = 1 THEN
      SI <- (P * R * T) / 100
      PRINT SI
    ELSE IF choice = 2 THEN
      CA <- P * (1 + R/100) ^ T
      PRINT CA
    ELSE IF choice = 3 THEN
      CA <- P * (1 + R/100) ^ T
      CI <- CA - P
      PRINT CI
    ELSE IF choice = 4 THEN
      SI <- (P * R * T) / 100
      Total <- P + SI
      PRINT Total
    ELSE IF choice = 5 THEN
      PRINT "Exiting"
    ELSE
      PRINT "Invalid choice"
    END IF
  UNTIL choice = 5
END
```

Step 5: Flowchart



Evaluation against Step 2 test cases: tracing each test case's menu choice and inputs through the decision boxes above yields the same results as the Expected Output(s) column, confirming the flowchart correctly represents the menu-driven logic.

Step 6: C Program

```

/* Q3: Menu-driven program
Option 1: Simple Interest      SI = (P * R * T) / 100
Option 2: Compound Amount     CA = P * (1 + R/100)^T
Option 3: Compound Interest   CI = CA - P
Option 4: Total Payable Amount (Simple Interest basis) Total = P + SI
*/
#include <stdio.h>

```

```

#include <math.h>

int main() {
    int choice;
    double p, r, t;

    while (1) {
        printf("\n--- FINANCIAL CALCULATOR ---\n");
        printf("1. Simple Interest (SI)\n");
        printf("2. Compound Amount (CA)\n");
        printf("3. Compound Interest (CI)\n");
        printf("4. Total Payable Amount (P + SI)\n");
        printf("5. Exit\n");
        printf("Enter choice (1-5): ");
        scanf("%d", &choice);

        if (choice == 5) {
            printf("Exiting...\n");
            break;
        }

        if (choice >= 1 && choice <= 4) {
            printf("Enter Principal (P): ");
            scanf("%lf", &p);
            printf("Enter Rate (R): ");
            scanf("%lf", &r);
            printf("Enter Time (T): ");
            scanf("%lf", &t);

            if (choice == 1) {
                double si = (p * r * t) / 100.0;
                printf("SI = %.2f\n", si);
            } else if (choice == 2) {
                double ca = p * pow((1.0 + r / 100.0), t);
                printf("CA = %.2f\n", ca);
            } else if (choice == 3) {
                double ca = p * pow((1.0 + r / 100.0), t);
                double ci = ca - p;
                printf("CI = %.2f\n", ci);
            } else if (choice == 4) {
                double si = (p * r * t) / 100.0;
                double total = p + si;
                printf("Total = %.2f\n", total);
            }
        } else {
            printf("Invalid choice! Try again.\n");
        }
    }

    return 0;
}

```

Step 7: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc9_3.c -lm      (compile)
./a.out              (execute)
```

All 3 test cases prepared in Step 2 were executed against the compiled program by selecting the corresponding menu option. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 8: Compilation and Execution Screenshot

```

Session Actions Edit View Help

(kali㉿kali)-[~]
$ nano hdoc9_3.c

(kali㉿kali)-[~]
$ cc hdoc9_3.c -lm

(kali㉿kali)-[~]
$ ./a.out

--- FINANCIAL CALCULATOR ---
1. Simple Interest (SI)
2. Compound Amount (CA)
3. Compound Interest (CI)
4. Total Payable Amount (P + SI)
5. Exit
Enter choice (1-5): 1
Enter Principal (P): 1000
Enter Rate (R): 5
Enter Time (T): 2
SI = 100.00

--- FINANCIAL CALCULATOR ---
1. Simple Interest (SI)
2. Compound Amount (CA)
3. Compound Interest (CI)
4. Total Payable Amount (P + SI)
5. Exit
Enter choice (1-5): 3
Enter Principal (P): 1000
Enter Rate (R): 10
Enter Time (T): 3
CI = 331.00

--- FINANCIAL CALCULATOR ---
1. Simple Interest (SI)
2. Compound Amount (CA)
3. Compound Interest (CI)
4. Total Payable Amount (P + SI)
5. Exit
Enter choice (1-5): 4
Enter Principal (P): 2000
Enter Rate (R): 10
Enter Time (T): 3
Total = 2600.00

--- FINANCIAL CALCULATOR ---
1. Simple Interest (SI)
2. Compound Amount (CA)
3. Compound Interest (CI)
4. Total Payable Amount (P + SI)
5. Exit
Enter choice (1-5): 5
Exiting...
```

➤ Conclusion

In this lab activity, menu-driven C programs were implemented to solve real-world mathematical, computational, and financial problems using modular control structures:

1. **Question 1** (Swapping Two Numbers): Demonstrated swapping using both a temporary variable and arithmetic operations without a temporary variable within an interactive menu loop.
2. **Question 2** (Natural Numbers Calculations): Calculated the sum of the first n natural numbers and the sum of their squares using direct formula evaluations, handling user input efficiently.
3. **Question 3** (Financial Calculations): Implemented calculations for Simple Interest, Compound Amount, Compound Interest, and Total Payable Amount, utilizing the `math.h` library for exponential operations.

All programs were compiled using `gcc` in a Linux terminal environment (linking the `math` library via `-lm` where required). All test cases executed as expected and verified against the manual trace tables.
